CSCE 411: Design and Analysis of Algorithms (Honors)

Week 12a: Space Complexity and Games

Date: November 10, 2026 *Samson Zhou*

1 Space Complexity

Last lecture, we distinguished the size of an input from the value it represents. We now ask how much memory an algorithm needs, as a function of that input size.

Example Suppose we solve SAT by trying all assignments to n Boolean variables.

- How many assignments are there? _____
- How many bits are needed to store one assignment? ____
- Do we need to keep all the assignments in memory at once?

The *space complexity* of an algorithm measures the largest amount of workspace it uses at any one time. We include counters and the recursion stack, but do not count the read-only input.

Definition. The class PSPACE consists of decision problems solvable by a deterministic algorithm that always halts and uses polynomial space.

Theorem 1.1. $\mathrm{P} \subseteq \mathrm{PSPACE}$.

Proof. How much workspace can an algorithm visit in a polynomial number of steps?

What about NP?

Theorem 1.2. NP $\subseteq$ PSPACE.

Proof. Let $Q \in$ NP. There is a polynomial-time verifier V and a polynomial p such that

$$x \in Q \iff V(x, y) \text{ accepts for some } y \text{ with } |y| \leq p(|x|).$$

The algorithm

- Enumerate all binary strings y of length at most $p(|x|)$, one at a time.
- For each y, run $V(x, y)$. Accept if it accepts.
- If no candidate is accepted, reject.

Account for the space What do we need to keep in memory? Which workspace can we reuse?

Question 1. *Why does the exponential number of possible certificates not force exponential space?*

A *We only need to store one candidate and one verifier computation at a time*

B *Every NP problem has only polynomially many certificates*

C *Exponential time always implies polynomial space*

2 Quantified Boolean Formulas

Recall that SAT asks whether there is an assignment that makes a Boolean formula true. We can write this as

$$\exists x_1 \exists x_2 \cdots \exists x_m \ \varphi(x_1, \ldots, x_m).$$

What changes if some variables must work for *both* possible values?

- $\exists x \, \varphi(x)$ is true if _____
- $\forall x \, \varphi(x)$ is true if _____

Play with a neighbor One player is True and chooses the variables marked $\exists$. The other is False and chooses the variables marked $\forall$. Read from left to right; each player sees all earlier choices. True wins if the resulting formula is true.

Question 2. *Who has a winning strategy in each game? Here $x \leftrightarrow y$ means that x and y have the same value.*

A $\forall x \exists y \ (x \leftrightarrow y)$

B $\exists y \forall x \ (x \leftrightarrow y)$

The quantified formula is true exactly when _____

Observation. An existential choice can depend on earlier universal choices, but cannot depend on choices that have not been made yet.

3 True Quantified Boolean Formula

An input to TQBF has the form

$$Q_1 x_1 Q_2 x_2 \cdots Q_m x_m \ \varphi(x_1, \ldots, x_m),$$

where every Q_i is either $\exists$ or $\forall$, and every variable is quantified. The decision problem asks whether the quantified formula is true.

TQBF =

Example Consider the formula

$$\exists x \forall y \exists z \ \big[(x \vee y) \wedge (y \leftrightarrow z)\big].$$

Does True have a winning strategy? Specify the first choice and how the last choice depends on False's move.

Evaluating the game tree Try both values of the next variable and recursively evaluate what remains.

- At an $\exists$ node, combine the two answers using _____
- At a $\forall$ node, combine the two answers using _____

How much space does the game tree need?

Theorem 3.1. TQBF $\in$ PSPACE.

Proof. Evaluate the tree using depth-first search. At each node, evaluate the first child, remember its answer, and then evaluate the second child.

Account for the space Let n be the length of the input formula and $m \leq n$ its number of variables.

- How deep is the recursion? _____
- What must we remember about the assignments on the current path?
- After evaluating the first child, how much of its subtree must we keep while evaluating the second child?

There can be 2^m leaves, but we do not store the whole tree. The current path, recursion stack, and workspace for evaluating φ all take polynomial space in n.

Observation. As with enumerating SAT assignments, a large number of possibilities does not mean that all possibilities must be stored at once.

PSPACE-completeness

A decision problem Q is *PSPACE-complete* if:

- $Q \in$ PSPACE; and
- Every problem in PSPACE has a polynomial-time reduction to Q.

Theorem 3.2. TQBF *is* _____

We proved that TQBF $\in$ PSPACE. We will use the hardness part as a known theorem, without giving the reductions.

Question 3. *Suppose someone finds a polynomial-time algorithm for* TQBF*. What follows?*

A *Only* TQBF *becomes easy*

B *Every problem in* PSPACE *can be solved in polynomial time*

C *Every decidable problem can be solved in polynomial time*

D *The halting problem becomes decidable*

Use the containments we have proved Under this hypothesis, fill in the relationships:

P ___ NP ___ PSPACE.

Explain where we used PSPACE-completeness.

We have seen that a problem can be solved using a modest amount of memory while taking a very large number of steps. Next, we will place an upper bound on that running time.

4 Exponential Time

The class EXPTIME consists of decision problems that can be solved by a deterministic algorithm in time $2^{p(n)}$ for some polynomial p, where n is the input length.

Examples of allowed time bounds

$$2^n, \qquad 2^{n^2}, \qquad 2^{n^3}.$$

Theorem 4.1. P $\subseteq$ EXPTIME.

Proof.

Question 4. *An algorithm takes 2^n steps to solve a decision problem Q. Does this prove that $Q \notin$ P?*

A *Yes, because 2^n is not polynomial*

B *No, because there might be a faster algorithm for Q*

C *No, because 2^n is polynomial*

Example A program waits for 2^n steps, then accepts if its input begins with 1 and rejects otherwise. It rejects the empty string. How quickly could we solve the same problem?

5 Relating Space and Time

A polynomial-space algorithm might take much longer than polynomial time. How long could it run before it must repeat itself?

A small example Suppose a deterministic machine has:

- 3 possible internal states;
- 4 work-tape cells, each containing one bit;
- a work-tape head pointing to one of the 4 cells.

For this example, its input head stays fixed.
How many different complete snapshots of this machine are possible?

Configurations A *configuration* records everything needed to determine the next step: the internal state, the tape contents, and all head positions. We consider a fixed input.

Question 5. *If a deterministic machine repeats a configuration before halting, what happens?*

A *It must halt at the next step*

B *It repeats the same computation and loops forever*

C *It may choose a different next step the second time*

Polynomial space gives an exponential time bound

Theorem 5.1. PSPACE $\subseteq$ EXPTIME.

Proof. Fix a deterministic decider and an input of length n. Suppose the machine uses at most $s(n) \geq 1$ work-tape cells, where s is polynomial.
For simplicity, use one work tape. The input head stays between the two input end-markers. Let q be the number of internal states and Γ the work-tape alphabet. Both are constants for this machine.

Count the configurations

- Internal state: _____
- Input-head position: _____
- Work-head position: _____
- Work-tape contents: _____

Therefore, the number of configurations is at most

$$q(n+2)s(n)|\Gamma|^{s(n)} \leq _____$$

for some polynomial p.
Why can the decider not repeat a configuration before halting?

Thus, its running time is bounded by the number of possible configurations, which is exponential in a polynomial in n.

$$\mathrm{P} \subseteq \mathrm{NP} \subseteq \mathrm{PSPACE} \subseteq \mathrm{EXPTIME}.$$

Which of these containments are strict? We will return to this question in the next lecture.

6 Games and Exponential Time

In TQBF, two players choose Boolean values, and each play has only one move per variable. What happens when a game can continue for many more moves?

A reachability game Two players, Alice and Bob, move a token between positions. Each position specifies whose turn it is and two possible next positions.

- Alice wins if the token reaches a goal position.
- Bob wins if the token never reaches a goal position.

Both players know the current position and all earlier moves. A strategy tells a player which move to make after each possible history.

Play with a neighbor The token begins at s. The goal is g. A position ℓ is a losing trap: both moves from ℓ return to ℓ.

Position	Whose turn?	First move	Second move
s	Alice	a	b
a	Bob	g	ℓ
b	Alice	g	ℓ
ℓ	Bob	ℓ	ℓ

Which first move lets Alice force a win? What happens if Alice chooses the other first move?

Observation. Having some sequence of moves that reaches the goal is different from having a strategy that wins against every response by the opponent.

Describing a large game

For the problem below, a position is a string of k bits. Thus there are _____ possible positions. We describe the rules compactly, instead of listing all positions and moves.

The input An instance consists of an initial position $s_0 \in \{0,1\}^k$ and Boolean circuits specifying:

- $O(s)$: whose turn it is at position s (0 for Alice, 1 for Bob);
- $G(s)$: whether s is a goal position;
- $N_0(s)$ and $N_1(s)$: the two possible next positions, each a k-bit string.

A Boolean circuit is a finite acyclic network of AND, OR, and NOT gates. We can evaluate it in time polynomial in its description length.

The rules At a goal position, Alice immediately wins. Otherwise, the player whose turn it is chooses one of the two next positions. If play continues forever without reaching a goal, Bob wins. The two next positions are allowed to be the same.
Definition. The decision problem GAMES asks whether Alice has a winning strategy from s_0.

GAMES =

To be clear. The input is the initial position and the circuits. Its length is not the number of positions in the game.

Theorem 6.1. GAMES *is* _____

This means that GAMES belongs to EXPTIME and every problem in EXPTIME has a polynomial-time reduction to GAMES. We will use hardness as a known theorem and prove membership next.

Finding the winning positions

Theorem 6.2. GAMES $\in$ EXPTIME.

Proof. Enumerate the 2^k positions. Initially, mark all goal positions as winning for Alice. Repeatedly apply the following rules:

- If it is Alice's turn, mark the position when:

- If it is Bob's turn, mark the position when:

In each round, use the marks from the previous round. Stop when a round adds no new marks. Answer YES exactly when s_0 is marked.

Why are the marked positions winning? Think about the round in which a position was first marked. Can Alice ensure that this round number decreases after each move, until the goal is reached?

What about an unmarked position? After the process stops, can Bob keep the token outside the marked set?

Running time There are at most ____ rounds that add new positions. Each round takes polynomial time in the size of the explicit position table and the circuit descriptions. If n is the total input length, then $k \leq n$, and the total running time is

$$(2^k + n)^{O(1)} = 2^{\text{poly}(n)}.$$

Observation. If the whole game graph were explicitly listed, this procedure would take polynomial time in the size of that graph. The compact representation is essential here.

Looking ahead We have algorithms that use exponential time. Can we prove that some decidable problem has no polynomial-time algorithm? Next lecture, we will return to the halting proof and add a clock.